Semana 33 — Conteúdo de 2 aulas

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Semana 33 — Conteúdo de 2 aulas

- **Aula 53:** Integração e Entrega Contínua (CI/CD)
- **Aula 54:** Prática: Pipeline CI/CD

Aula 53 — Integração e Entrega Contínua (CI/CD)

Objetivos da Aula

- Compreender **CI**, **CD** e a diferença entre entrega e deploy.
- Conhecer as **etapas de um pipeline**.
- Identificar **ferramentas** de CI/CD.
- Entender boas práticas de pipelines.

Integração Contínua (CI)

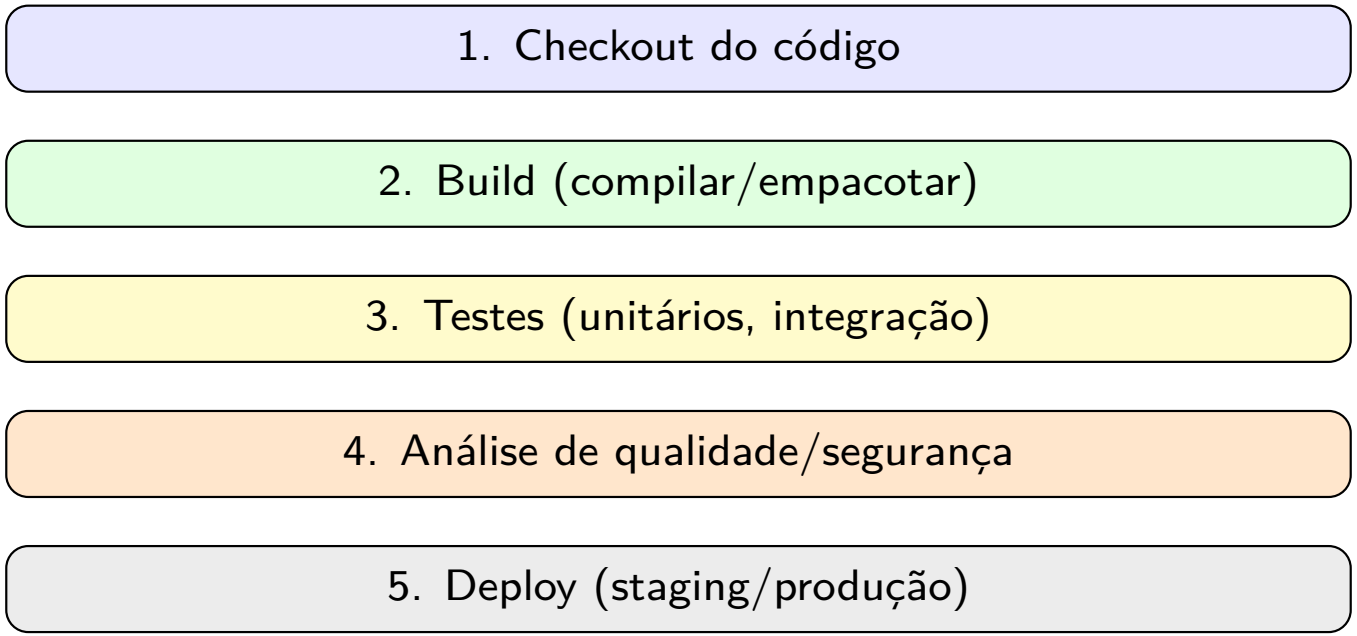
- Integrar o código **frequentemente** (várias vezes ao dia).
- Cada integração dispara **build e testes automatizados**.
- Detecta erros **cedo**.
- Requer **versionamento** (Git) e **testes**.

Entrega × Deploy Contínuo

- **Continuous Delivery:** pronto para produção, **deploy manual**.
- **Continuous Deployment: deploy automático** em produção.
- A escolha depende de **risco e maturidade**.
- Ambos dependem de **testes confiáveis**.

Etapas de um Pipeline

1. **Checkout** do código.
2. **Build** (compilar/empacotar).
3. **Testes** (unitários, integração).
4. **Análise** de qualidade/segurança.
5. **Deploy** (staging/produção).



Pipeline como Código

```yaml
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "compilar"
      - run: echo "testar"
      - run: echo "publicar"
```

- O pipeline é **versionado** com o código.
- Reproduzível e revisável.

Ferramentas de CI/CD

- **GitHub Actions, GitLab CI**.
- **Jenkins** (servidor de automação).
- **AWS CodePipeline, Azure DevOps**.
- Integração com **contêineres e IaC**.

Estratégias de Deploy

- **Rolling:** substituição gradual.
- **Blue-green:** dois ambientes, troca de tráfego.
- **Canary:** liberar para uma fração dos usuários.
- Reduzem **risco** de indisponibilidade.

Boas Práticas

- **Testes automatizados** confiáveis.
- **Pipelines rápidos** e feedback curto.
- **Segredos** gerenciados com segurança.
- **Monitoramento** pós-deploy.
- **Rollback** fácil.

Tip

Um bom pipeline é **rápido e confiável**: é a espinha dorsal da entrega contínua. Deploy sem teste é risco, não agilidade.

Resumo da Aula

- **CI** integra e testa continuamente.
- **CD** entrega/deploya automaticamente.
- Pipelines têm etapas de **build, teste e deploy**.
- Estratégias como **blue-green e canary** reduzem risco.

Atividade Rápida

- Diferencie **entrega** e **deploy** contínuo.
- Liste as etapas de um **pipeline**.
- Explique a estratégia **blue-green**.

Referências

- GitHub Actions / GitLab CI — *Documentação*.
- *Continuous Delivery* (Humble & Farley).

Aula 54 — Prática: Pipeline CI/CD

Objetivos da Aula

- Criar um **repositório** com uma aplicação simples.
- Configurar um **pipeline de CI/CD**.
- Automatizar **build, teste e deploy**.
- Consolidar DevOps na prática.

Preparação

- Conta no **GitHub** (ou GitLab).
- Aplicação simples (ex.: API, página, script).
- Noções de **Git** (commit, push).
- Ambiente onde publicar (ex.: contêiner, hospedagem).

Passo 1 — Repositório

- Criar o repositório e enviar o código.
- Adicionar **testes** automatizados.
- Definir a **branch principal**.
- Documentar no **README**.

Passo 2 — Configurar o Pipeline

- Criar `.github/workflows/ci.yml` (exemplo).
- Definir **gatilhos** (push, pull request).
- Etapas: **checkout** → **setup** → **build** → **test**.
- Rodar em um ambiente limpo.

Passo 3 — Testes Automatizados

- Executar a suíte de testes no pipeline.
- Falhar o pipeline se um teste falhar.
- Adicionar verificação de **formatação/lint**.
- Publicar **relatórios** de teste.

Passo 4 — Deploy

- Adicionar etapa de **empacotamento** (ex.: imagem Docker).
- Publicar no registro de imagens.
- Deploy em **staging** e depois **produção**.
- Configurar **segredos** com segurança.

Passo 5 — Validação

- Fazer um **commit** e acompanhar o pipeline.
- Introduzir um **erro** e ver o pipeline falhar.
- Corrigir e ver o pipeline **passar**.
- Observar o deploy.

Verificação da Prática

- ☐ Repositório com testes.
- ☐ Pipeline configurado e disparando.
- ☐ Build e testes automáticos.
- ☐ Deploy automatizado (staging/produção).

Tip

Mantenha o pipeline **enxuto e rápido**. Feedback em poucos minutos acelera todo o ciclo de desenvolvimento.

Problemas Comuns

- Pipeline lento $\rightarrow$ cachear dependências.
- Falha por segredo exposto $\rightarrow$ usar **secrets**.
- Testes instáveis $\rightarrow$ revisar ambientes.
- Deploy quebrando $\rightarrow$ validar em **staging** primeiro.

Resumo da Aula

- Criamos um **pipeline** do zero.
- Automatizamos **build, teste e deploy**.
- Usamos **segredos** e ambiente limpo.
- Próxima aula: **atividade sobre DevOps**.

Referências

- GitHub Actions — *Documentação*.
- Material das aulas 64 e 65.